

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



HorarioScript: Analizador Léxico para Horarios Académicos

PONDERACIÓN: 25 pts

Tiempo estimado: 20 hrs

Índice

1.	MARCO FORMATIVO	3
1.1.	Valor	3
1.2.	Competencia(s)	3
1.3.	Habilidad(es) blandas a formar	3
2.	Resultado del Aprendizaje	3
2.1.	Objetivo SMART	3
3.	Resumen Ejecutivo	4
4.	Enunciado del Proyecto	5
4.1	Descripción del problema o necesidad a resolver	5
4.2	Alcance del proyecto	5
4.3	Recursos y herramientas a utilizar	6
4.4	Entregables	6
4.5	Estructura del Lenguaje HorarioScript	7
4.6	Ejemplo de Archivo de Entrada	9
4.7	Reportes a Generar	9
4.8	Tabla de Errores Léxicos	10
5.	Material de apoyo	11
6.	Metodología	11
7.	Cronograma	12
8.	Rúbrica de Calificación	12
8.1	Requisitos para optar a la calificación	12
8.2	Detalle de la Calificación	14
8.3	Comentarios Generales	18

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Excelencia	Los estudiantes demostrarán excelencia técnica al implementar desde cero, en Python, un analizador léxico funcional y preciso para el lenguaje HorarioScript, aplicando los principios de teoría de autómatas y lenguajes formales con rigor profesional. La calidad del código, la documentación y la interfaz gráfica reflejarán el nivel de compromiso con los más altos estándares académicos.

1.2. Competencia(s)

Con la elaboración de este proyecto usted adquirirá la siguiente competencia:

Competencia General: Diseñar, implementar y evaluar soluciones computacionales aplicando los fundamentos teóricos y prácticos de las ciencias de la computación para resolver problemas reales en el ámbito del desarrollo de software. **Competencia Específica:** Aplicar los conceptos de teoría de autómatas, lenguajes formales y análisis léxico para desarrollar, en Python, un analizador léxico que procese el lenguaje de dominio específico HorarioScript, implementando el AFD manualmente sin librerías de expresiones regulares ni generadores automáticos.

1.3. Habilidad(es) blandas a formar

El proyecto le permitirá desarrollar las siguientes habilidades:

Autogestión del tiempo: planificar las 4 fases del proyecto en 4 semanas, dedicando entre 5 y 6 horas semanales.

Responsabilidad y compromiso: autoría íntegra del código, la documentación y los casos de prueba, sin usar herramientas generadoras automáticas de analizadores léxicos.

Resolución de problemas: manejar ambigüedades del lenguaje (por ejemplo, distinguir un código de aula de una hora) e implementar recuperación de errores robusta.

Pensamiento crítico: evaluar distintas estrategias para representar los estados del AFD en Python y justificar las decisiones de diseño en el manual técnico.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Los estudiantes diseñarán e	El analizador reconocerá	Utilizando Python 3.10+	Fortalece las competencias	Completar en 4 semanas:

implementarán desde cero, sin el módulo re ni generadores automáticos, un analizador léxico en Python con interfaz gráfica en Tkinter para el lenguaje HorarioScript, capaz de reconocer cursos, catedráticos, aulas y bloques de clase, detectar errores léxicos y choques de horario.	correctamente al menos el 95% de los tokens definidos en los casos de prueba, detectará el 100% de los choques de horario simulados, y generará 3 reportes HTML exportables (horario semanal, carga de catedráticos y estadístico general).	con Tkinter para la GUI, AFD implementado manualmente con funciones de transición de estados explícitas, Git/GitHub para control de versiones y Graphviz para los diagramas del AFD.	en teoría de autómatas, diseño de analizadores léxicos y desarrollo de interfaces gráficas en Python, preparando al estudiante para el Proyecto 2 (análisis sintáctico) del mismo curso.	Semana 1 diseño del lenguaje y AFD, Semana 2 motor léxico, Semana 3 integración con la GUI y reportes, Semana 4 pruebas y documentación.
---	---	--	--	--

3. Resumen Ejecutivo

HorarioScript es un proyecto de desarrollo de software cuyo objetivo es construir, en Python, un analizador léxico con interfaz gráfica (Tkinter) capaz de leer, interpretar y reportar horarios académicos a partir de archivos de texto con extensión .hor, estructurados bajo las reglas del mini-lenguaje HorarioScript. El proyecto se enfoca en la implementación manual de un Autómata Finito Determinista (AFD) para el análisis léxico, sin recurrir al módulo re ni a herramientas automáticas de generación léxica.

La solución integra tres componentes principales: el motor léxico (AFD implementado a mano en Python), la interfaz gráfica de escritorio (Tkinter), y el generador de tres reportes HTML (horario semanal por sección, carga de catedráticos y estadístico general del ciclo). El sistema debe además detectar automáticamente choques de horario: casos en que un mismo catedrático o una misma aula tienen dos clases asignadas en el mismo día y bloque horario. El proyecto exige que el estudiante comprenda en profundidad la teoría de autómatas finitos y sea capaz de traducirla a código Python eficiente, manejando correctamente todos los tipos de tokens del lenguaje, incluyendo códigos con guión, literales de hora y enumeraciones de días.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

Los sistemas de programación académica manejan información estructurada sobre cursos, catedráticos, aulas y bloques horarios. Un error en la asignación de horarios —por ejemplo, un mismo catedrático programado en dos aulas a la misma hora— genera conflictos que afectan directamente la operación de una facultad.

El problema que este proyecto busca resolver es la necesidad de una herramienta interactiva que permita comprender cómo se procesa léxicamente un lenguaje formal de dominio académico. La herramienta debe interpretar archivos .hor escritos en el mini-lenguaje HorarioScript, detectar y reportar con precisión cualquier error léxico, identificar choques de horario, y generar reportes visuales que faciliten la revisión de la programación académica.

4.2 Alcance del proyecto

Alcance obligatorio:

- Lenguaje HorarioScript con al menos 10 tipos de token: palabras reservadas (HORARIO, CURSOS, CATEDRATICOS, AULAS, CLASES), códigos con guión (LFP-0796, DOC-001, A-101), literales de hora (HH:MM), enteros, cadenas de texto, enumeración de días (LUNES...SABADO), enumeración de categoría de catedrático (TITULAR, INTERINO, AUXILIAR), delimitadores estructurales y comentarios de línea.
- AFD implementado manualmente en Python con la clase AnalizadorLexico y el método siguiente_token(). Prohibido el uso del módulo re y de funciones de alto nivel de cadenas (split, find, etc.) para la tokenización principal; solo se permite el acceso carácter a carácter (indexación) y la lectura del archivo.
- Tabla de tokens: número, lexema, tipo de token, línea y columna, con precisión del 100%.
- Tabla de errores léxicos: número, lexema inválido, tipo de error, descripción y posición exacta. El análisis debe continuar tras cada error, sin detenerse (modo pánico).
- Detección de choques de horario: mismo catedrático o misma aula, mismo día y bloque horario traslapado.
- Tres reportes HTML con CSS embebido (descritos en la sección 4.6).
- Diagrama Graphviz (código DOT) con la jerarquía del horario y las relaciones curso-catedrático-aula.
- Interfaz gráfica en Tkinter con todos los paneles requeridos.

Alcance opcional:

- Resaltado de sintaxis en tiempo real en el área de texto.
- Exportación de la tabla de tokens a CSV o JSON.
- Estadísticas del análisis léxico: frecuencia de tokens por tipo, tiempo de análisis en ms.
- Soporte para analizar múltiples archivos .hor en una misma sesión.

- Sugerencia automática de bloques horarios libres para reprogramar una clase en conflicto.

4.3 Recursos y herramientas a utilizar

Tipo (Obligatorio / opcional)	Categoría (Software / hardware / Plataforma / Etc)	Descripción
Obligatorio	Software - Lenguaje	Python 3.10 o superior.
Obligatorio	Software - GUI	Tkinter (incluida en la biblioteca estándar de Python).
Obligatorio	Plataforma	Git y GitHub para control de versiones.
Obligatorio	Herramienta de diseño	Graphviz, para los diagramas del AFD en el manual técnico.
Opcional	IDE	VS Code o PyCharm.
Opcional	Herramienta de diseño	Draw.io o Lucidchart, para diagramas UML complementarios.

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Código Fuente	Código completo en Python, organizado en módulos/clases (Token, AnalizadorLexico, GestorErrores, GeneradorReportes y la clase principal de la interfaz Tkinter), debidamente comentado y siguiendo buenas prácticas de programación orientada a objetos.
Manual Técnico (PDF)	Arquitectura del sistema, diagrama de clases, diagrama del AFD con todos los estados y transiciones (Graphviz), tabla de transiciones, descripción del algoritmo de tokenización y de la lógica de detección de choques de horario, y justificación de decisiones de diseño.

Manual de Usuario (PDF)	Guía paso a paso con capturas de pantalla: cómo ejecutar la aplicación, la estructura del archivo .hor, cómo cargar y analizar un archivo, cómo interpretar la tabla de tokens y el reporte de errores, y cómo abrir y navegar cada uno de los 3 reportes HTML generados.										
Casos de Prueba (PDF)	Documento con al menos 8 casos ejecutados: archivo válido con las 4 secciones, errores de token inválido, hora fuera de rango, día no reconocido, código mal formado, comentario sin cerrar, choques de horario simulados y casos borde. Cada caso con entrada, resultado esperado y resultado obtenido.										
Repositorio GitHub	Repositorio privado con el nombre LFP_<SECCIÓN>_2S2026, que contenga la carpeta del proyecto Proyecto1 con todo el código fuente en Python, así como un archivo README.md con las instrucciones de ejecución y ejemplos de uso. Agregar a su tutor al repositorio. <table border="1"> <thead> <tr> <th>Sección</th><th>Usuario Tutor</th></tr> </thead> <tbody> <tr> <td>A</td><td>Adriluuu</td></tr> <tr> <td>B+</td><td>SamuelAguilar18</td></tr> <tr> <td>B-</td><td>JoseOre12</td></tr> <tr> <td>N</td><td>JorgeP3</td></tr> </tbody> </table>	Sección	Usuario Tutor	A	Adriluuu	B+	SamuelAguilar18	B-	JoseOre12	N	JorgeP3
Sección	Usuario Tutor										
A	Adriluuu										
B+	SamuelAguilar18										
B-	JoseOre12										
N	JorgeP3										

4.5 Estructura del Lenguaje HorarioScript

El archivo de entrada con extensión .hor sigue una estructura jerárquica donde el bloque raíz HORARIO contiene cuatro secciones: CURSOS, CATEDRATICOS, AULAS y CLASES. La siguiente tabla define los tipos de token que el AFD debe reconocer, con su patrón/regla y un ejemplo de lexema.

Token / Elemento	Tipo	Descripción / Regla	Ejemplo
HORARIO, CURSOS, CATEDRATICOS, AULAS, CLASES	Palabra reservada de bloque	Delimitan las secciones principales del archivo. Cada una aparece exactamente una vez.	HORARIO { ... }
curso, catedratico, aula, clase	Palabra reservada de elemento	Definen un elemento dentro de su sección correspondiente.	curso: "Lenguajes Formales" [...]
con, en	Palabra reservada de relación	Conectan una clase con su catedrático y su aula.	clase: "LFP-0796" con "DOC-001" en "A-101" [...]

CODIGO	Alfanumérico con	Letras + guión +	"LFP-0796", "DOC-
	guión	dígitos. Identifica cursos, catedráticos y aulas.	001", "A-101"
CADENA	Literal de texto	Texto entre comillas dobles. Puede contener cualquier carácter excepto salto de línea sin escapar.	"Lenguajes Formales y de Programación"
HORA	Literal de hora	Formato HH:MM en 24 horas (06:00 - 21:00, horario institucional).	07:00, 18:40
ENTERO	Literal numérico	Uno o más dígitos consecutivos. Usado en capacidad y créditos.	40, 4
DIA	Enumeración	LUNES MARTES MIERCOLES JUEVES VIERNES SABADO.	día: MARTES
CATEGORIA	Enumeración	TITULAR INTERINO AUXILIAR.	categoría: TITULAR
SIMBOLO	Carácter de puntuación	Caracteres con significado sintáctico: { } [] : , ; . Cada uno es un token independiente.	{ } [] : ,
COMENTARIO_LIN EA	Comentario	Inicia con ## y abarca hasta el final de la línea (\n o EOF).	## Ciclo 2026-1

Reglas de prioridad y manejo especial:

- Palabras reservadas vs. códigos: el AFD primero consume la secuencia alfanumérica completa; si coincide con una palabra reservada de la lista, el token es esa palabra reservada, si no, se evalúa como CODIGO (si sigue el patrón letras-guión-dígitos).
- Hora vs. código: al leer dos dígitos seguidos de ':', el AFD reconoce HORA; si en cambio sigue un guión, reconoce el inicio de un CODIGO.
- Comentario de línea: la secuencia ## inicia el comentario; todo hasta \n o EOF

pertenece al mismo token y no se reporta como error aunque contenga caracteres fuera del alfabeto.

- Cadenas sin cerrar: si se abre " y se alcanza fin de línea o EOF sin el cierre, se emite un ERROR_LEXICO con el mensaje "Cadena sin cerrar" y la posición de apertura.
- Caracteres no reconocidos: cualquier carácter que no inicie ningún patrón válido (por ejemplo @, %, ~) se reporta como ERROR_LEXICO con el mensaje "Carácter no reconocido: '<char>'" y su posición exacta.
- Espacios en blanco y saltos de línea: son delimitadores silenciosos. No generan token, pero actualizan el contador de línea/columna.

4.6 Ejemplo de Archivo de Entrada

A continuación se presenta un archivo de entrada válido de ejemplo (horario_valido.hor):

```
## Horario Lenguajes Formales - Segundo Semestre 2026
HORARIO {
  CURSOS {
    curso: "Lenguajes Formales y de Programacion" [codigo: "LFP-0796", creditos:
4],
    curso: "Bases de Datos 2" [codigo: "BD2-0812", creditos: 5],
  };
  CATEDRATICOS {
    catedratico: "Otto Rodriguez" [codigo: "DOC-001", categoria: TITULAR],
    catedratico: "Vivian Campos" [codigo: "DOC-002", categoria: INTERINO],
  };
  AULAS {
    aula: "A-101" [capacidad: 40, edificio: "T-3"],
    aula: "LAB-3" [capacidad: 25, edificio: "T-5"],
  };
  CLASES {
    clase: "LFP-0796" con "DOC-001" en "A-101" [dia: LUNES, inicio: 07:00, fin:
08:40, seccion: "N"],
    clase: "BD2-0812" con "DOC-002" en "LAB-3" [dia: MARTES, inicio: 10:20, fin:
12:00, seccion: "A"],
  };
};
```

Este mismo archivo, si se modificara para que "DOC-001" tuviera una segunda clase el LUNES de 07:00 a 08:40 en otra aula, debería ser detectado por el sistema como un choque de horario (ver sección 4.7, Reporte 1).

4.7 Reportes a Generar

El sistema debe generar automáticamente tres reportes HTML al finalizar el análisis léxico, aprovechando la información extraída de los tokens. Cada reporte se guarda como un archivo HTML independiente con CSS embebido y debe poder abrirse en cualquier navegador.

Reporte 1 — Horario Semanal por Sección

Objetivo: presentar la agenda semanal (Lunes a Sábado) con los bloques horarios de cada sección, mostrando curso, catedrático y aula. Las celdas donde el mismo catedrático o la misma aula tienen dos clases traslapadas en el mismo día y hora deben resaltarse con fondo rojo y una marca de advertencia, exactamente igual que se detectó el choque.

- Contenido mínimo por celda: curso, catedrático y aula (o vacío si no hay clase en ese bloque).
- Estado de cada bloque: CONFIRMADO (fondo verde) o CHOQUE DE HORARIO (fondo rojo).
- El reporte debe generarse por cada sección presente en el archivo, o consolidado si el archivo no distingue secciones.

Reporte 2 — Carga de Catedráticos

Objetivo: mostrar el perfil de cada catedrático registrado, indicando su categoría, el total de horas semanales asignadas, el número de cursos y secciones que atiende, y un indicador visual del nivel de carga. Los umbrales son sugeridos (BAJA: 1-4 horas, NORMAL: 5-10, ALTA: 11-15, SATURADA: 16+) y el estudiante puede ajustarlos con justificación en el manual técnico.

- Nombre del catedrático, código único y categoría (del bloque CATEDRATICOS).
- Total de horas semanales asignadas (suma de la duración de cada bloque en CLASES).
- Número de cursos y de secciones distintas que atiende.
- Nivel de carga con color diferenciado: BAJA (azul), NORMAL (verde), ALTA (naranja), SATURADA (rojo).

Reporte 3 — Estadístico General del Ciclo

Objetivo: proveer un resumen ejecutivo con las estadísticas globales del horario procesado. Se divide en un panel de indicadores clave (KPIs) y una tabla de ocupación de aulas con una barra de progreso visual.

- Total de cursos, catedráticos, aulas y clases programadas.
- Número de choques de horario detectados.
- Catedrático con mayor carga de horas y aula con mayor ocupación.
- Promedio de horas semanales por catedrático.
- Tabla de ocupación por aula (aula, clases asignadas, % de ocupación sobre el total de bloques disponibles); las aulas con ocupación mayor al 80% deben resaltarse en rojo.

4.8 Tabla de Errores Léxicos

Cuando el archivo .hor contiene tokens inválidos, el analizador debe acumular todos los errores durante el recorrido completo del archivo (modo de recuperación de errores) y mostrarlos al finalizar, tanto en la interfaz gráfica como en un reporte HTML adicional. El sistema no debe detenerse ante el primer error; debe continuar el análisis para reportar todos los problemas en una sola pasada.

Tipo de Error	Descripción y mensaje sugerido
CARACTER_NO_RECONOCIDO	El carácter no pertenece al alfabeto del lenguaje. Mensaje: "Carácter no reconocido: '<char>' en línea L, columna C".
CADENA_SIN_CERRAR	Se abrió " y se llegó al fin de línea o EOF sin el cierre. Mensaje: "Cadena sin cerrar iniciada en línea L, columna C".
HORA_FUERA_DE_RANGO	La hora leída no está en formato HH:MM válido o excede el rango institucional 06:00-21:00. Mensaje: "Hora fuera de rango en

	línea L, columna C".
DIA_NO_RECONOCIDO	El valor del atributo dia no coincide con ninguno de los seis días válidos. Mensaje: "Día no reconocido: '<valor>' en línea L, columna C".
CODIGO_MAL_FORMADO	El código no sigue el patrón de Letras + guión + dígitos. Mensaje: "Código mal formado: '<valor>' en línea L, columna C".

5. Material de apoyo

Categoría (Manual / Documentación oficial/ Ejemplo / Etc)	Link	Descripción
Manual	N/A	Compilers: Principles, Techniques, and Tools (Dragon Book) — Aho, Sethi, Ullman. Capítulos 2 y 3. Fundamentos de análisis léxico y AFD.
Documentación oficial	https://docs.python.org/3/library/tkinter.html	Documentación oficial de Tkinter.
Documentación oficial	https://graphviz.org/doc/info/lang.html	Referencia del lenguaje DOT de Graphviz.
Ejemplo	N/A	Ejemplos de archivos .hor incluidos en la sección 4.6 de este documento.

**El link es opcional para una mejor guía del material de apoyo

6. Metodología

A continuación se presenta una metodología de la lógica que puedes seguir para elaborar el presente proyecto:

Pasos a seguir:

1. Fase 1: Análisis del Lenguaje y Diseño del AFD (Semana 1)
Definir formalmente la gramática léxica de HorarioScript: alfabeto, tipos de token, patrones y errores. Diseñar el diagrama completo del AFD con todos sus estados,

transiciones y estados de aceptación. Diseñar la arquitectura del sistema (clases, responsabilidades y relaciones).

2. Fase 2: Implementación del Motor Léxico (Semana 2)

Implementar la clase Token, la clase AnalizadorLexico con siguiente_token() basado en el AFD diseñado, y el GestorErrores para acumular errores y permitir que el análisis continúe. Verificar el funcionamiento desde consola con los casos de prueba.

3. Fase 3: Integración con la GUI y Generación de Reportes (Semana 3)

Diseñar e implementar la interfaz gráfica en Tkinter con todos los paneles requeridos. Integrar el motor léxico con la GUI para carga de archivos y visualización de resultados. Implementar el GeneradorReportes para los 3 reportes HTML y el código DOT de Graphviz.

4. Fase 4: Pruebas, Corrección y Documentación (Semana 4)

Ejecutar los casos de prueba y documentar los resultados. Corregir errores y verificar la cobertura del AFD con casos borde. Redactar el Manual Técnico y el Manual de Usuario. Organizar el repositorio GitHub y verificar que el proyecto corre desde cero siguiendo el README.

Recomendaciones

Evita la tentación de usar el módulo re “solo para probar rápido”: una vez usado, es difícil justificar el AFD manual ante el auxiliar en la calificación. Prueba tu analizador de forma incremental, token por token, antes de integrarlo con la GUI.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	01/09/2026	01/09/2026
Elaboración	01/09/2026	25/09/2026
Calificación	26/09/2026	26/09/2026

8. Rúbrica de Calificación

En esta sección se aborda todo lo relacionado a la calificación, lo que permite claridad para el estudiante y el tutor académico de los aspectos a evaluar y los mismos que conocen desde el inicio por lo que todo proyecto debe incluirlos desde el inicio.

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Tema	Descripción	Cumple (Sí/No)
Tecnología establecida	El desarrollo debe realizarse en Python 3.10+ con interfaz gráfica en Tkinter. No se permite el uso del módulo re ni de funciones de alto nivel de cadenas (split, find, etc.) para la tokenización principal, ni librerías generadoras de analizadores léxicos (ply o similares).	
Herramientas requeridas	El proyecto debe incluir control de versiones con Git, repositorio en GitHub con commits organizados y README completo.	
Documentación obligatoria	Manual Técnico, Manual de Usuario y Casos de Prueba en formato PDF o MD.	
Funcionalidad mínima	Cargar archivos .hor, reconocer al menos 10 tipos de token mediante AFD, generar tabla de tokens y tabla de errores con posición exacta, detectar choques de horario y generar los 3 reportes HTML.	

8.2 Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	40				
1.1	Documentación Técnica (Manual Técnico)	10	Descripción	El manual técnico está completo y profesional. Incluye arquitectura del sistema, diagrama de clases, diagrama del AFD con todos los estados y transiciones (Graphviz), tabla de transiciones, y justificación sólida de las decisiones de diseño.	El manual técnico está incompleto, carece de diagramas UML o del AFD, los algoritmos no están explicados adecuadamente, o la presentación es desorganizada.	
			Rango del punteo	10-6/	5-0/	
1.2	Documentación de Usuario (Manual de Usuario)	8	Descripción	El manual de usuario es completo, claro y profesional. Incluye instrucciones de ejecución, capturas de pantalla de la GUI y de los 3 reportes generados, y	El manual de usuario está incompleto, sin capturas de los reportes, o las instrucciones son confusas o insuficientes para	

				explicación de cómo interpretar tokens y errores.	usar la aplicación.	
			Rango del punteo	8-5/	4-0/	
1.3	Repositorio GitHub	8	Descripción	El repositorio está bien organizado, con commits frecuentes y descriptivos, README completo con instrucciones de ejecución y .gitignore apropiado para Python.	El repositorio tiene pocos commits o poco descriptivos, README incompleto o ausente, o falta .gitignore.	
			Rango del punteo	8-5/	4-0/	
1.4	Casos de Prueba y Documentación de Testing	8	Descripción	Incluye al menos 8 casos de prueba bien documentados que cubren código válido, distintos errores léxicos, casos borde y choques de horario, cada uno con entrada, resultado esperado y obtenido.	Los casos de prueba son insuficientes (menos de 8), están mal documentados, o no cubren escenarios de error o de choques de horario.	
			Rango del punteo	8-5/	4-0/	
1.5	Presentación	6	Descripción	La presentación es profesional, clara y bien estructurada, cubriendo introducción,	La presentación está incompleta, sin demostración	

				demonstración en vivo y explicación técnica del AFD.	funcional o sin explicación técnica del AFD.	
			Rango del punteo	6-4/	3-0/	
	Sub-Total de Puntos					
2	Conocimientos	60				
2.1	Carga y Procesamiento de Archivos .hor	6	Descripción	Carga correctamente archivos .hor, valida que el archivo exista y sea legible, y actualiza los contadores de línea/columna con precisión al encontrar saltos de línea y tabulaciones.	No carga archivos correctamente o los contadores de línea/columna son incorrectos con frecuencia.	
			Rango del punteo	6-4/	3-0/	
2.2	Reconocimiento de Tokens mediante AFD	15	Descripción	El AFD implementado manualmente en Python reconoce al menos el 95% de los tokens en los casos de prueba, incluyendo códigos, horas, días y categorías. No utiliza el módulo re ni funciones de alto nivel de cadenas.	Reconoce menos del 80% de los tokens, no distingue correctamente entre tipos similares (código vs. hora), o utiliza re/funciones de alto nivel para tokenizar.	

			Rango del punteo	15-9/	8-0/	
2.3	Tabla de Tokens	8	Descripción	Genera correctamente los campos número, lexema, tipo, línea y columna para todos los tokens reconocidos, visualizados en la interfaz gráfica con precisión del 100%.	Faltan campos, las posiciones son incorrectas con frecuencia, o la tabla no se muestra correctamente en la interfaz.	
			Rango del punteo	8-5/	4-0/	
2.4	Detección y Reporte de Errores Léxicos	8	Descripción	Detecta las categorías de error descritas en la sección 4.8, reporta la posición exacta y un mensaje descriptivo, y continúa el análisis tras cada error (recuperación de errores).	No detecta alguna categoría de error, las posiciones son incorrectas, o el análisis se detiene ante el primer error.	
			Rango del punteo	8-5/	4-0/	
2.5	Detección de Choques de Horario	10	Descripción	Detecta correctamente todos los choques de horario (mismo catedrático o misma aula, mismo día y bloque traslapado) y los resalta en el Reporte 1.	No detecta choques de horario, los detecta de forma incompleta, o genera falsos positivos con frecuencia.	
			Rango del punteo	10-6/	5-0/	
2.6	Interfaz Gráfica en Tkinter	8	Descripción	La interfaz en Tkinter es funcional e intuitiva: permite cargar el archivo,	Faltan componentes de la interfaz, los botones no	

				ejecutar el análisis, y visualizar la tabla de tokens, la tabla de errores y el acceso a los 3 reportes.	funcionan correctamente, o no se pueden visualizar los resultados del análisis.	
			Rango del punteo	8-5/	4-0/	
2.7	Funcionalidades Opcionales	5	Descripción	Implementa al menos una funcionalidad opcional de la sección 4.2 de manera correcta y funcional, agregando valor real al proyecto.	No implementa funcionalidades opcionales, o las implementadas están incompletas o no funcionan correctamente.	
			Rango del punteo	5-3/	2-0/	
	Sub-Total de Puntos					
	Total					

8.3 Comentarios Generales
